

NYC / AI AGENT SECURITY SUMMIT

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz

Principal Engineer and Head of DevRel at Incredibuild.com

October 21, 2026

The check is green. The parser is still broken.

```
task check          PASS
```

```
external case "1,,3"  REJECT expected  
actual parser output  [1, 3]
```

Would you accept the patch?

Recorded Linux isolation run

The job we asked for

Fix the parser.
Publish the task report.

Four independent experiments. A fresh reset around the same job.

Identity Files Data release Acceptance

Who can stop an unwanted action?

Worker controls: its code, workspace and requests.

Controller owns: launch policy, service permissions, verification and publication.

Synthetic credentials and data. A loopback service. No live model or external target.

Recorded Linux isolation run

01 / It can read another project's data

```
request: GET /projects/other/private
```

runner credential	200 + synthetic fixture
task credential	403
task input	200
expired credential	403

The service honored the credential we supplied.

Recorded Linux isolation run

Authority for this task

Launch: clean environment and closed extra descriptors.

Grant: task resource, operation, audience and expiry.

Enforcement: a service or broker outside the worker.

A stronger process boundary still needs narrower credentials.

02 / It can change another job's state

worker writes /shared/next-job.json

writable bind	host fixture changes
read-only bind	write denied; fixture unchanged
/output	patch remains writable

Observed with real bind mounts

Read this input. Write this output.

Resource

Worker access

Source input

Read only

Task output

Writable

Shared job state

Read only in this lab

Controller files

Outside the mount set

Production deployments can omit shared state entirely.

03 / It can send data to another account

same upload service

/accounts/team/reports intended recipient

/accounts/other/uploads synthetic private data

The hostname stayed the same. The recipient changed.

Recorded Linux isolation run

A report with a known recipient

Worker: requests one fixed report operation.

Broker: constructs the body and selects the team account.

Checks: rejects extra recipient, payload and URL fields.

Direct host-loopback probe: unreachable

04 / It can change what we call success

```
# Worker rewrites the workspace checker:  
print("PASS")  
raise SystemExit(0)
```

Now the opening makes sense: the worker rewrote the checker.

Recorded Linux isolation run

The original bug meets an independent check

Input	Buggy parser	Fixed parser
1,2,3	Accept	Accept
1,,3	Accept	Reject
Empty input	Accept	Reject
-2, 3	Accept	Accept
x	Reject	Reject

Recorded Linux isolation run

A passing result belongs to one artifact

Rejected artifact	Denied
Artifact swapped after verification	Denied
Authentic approval from a stale run	Denied
Verifier identity changed	Denied
Replayed approval	Denied
Verified artifact, current run	Accepted

Recorded Linux isolation run

The record needs an independent reference

The worker can rebuild a chain it owns.

The controller retains the original head. Rewritten history disagrees.

Recorded Linux isolation run

Back to the original job

1. Worker reads input and writes the fixed parser.
2. External verifier checks the frozen artifact.
3. Gate accepts exactly those bytes.
4. Broker publishes the fixed task report.

1,3 now returns REJECT. The report reaches the team.

Recorded Linux isolation run

What did we give it?

Question

What can execute and reach the host?

What can read and change files?

What can act and send data?

What can declare success?

Enforcing owner

Runtime

Host mount policy

Service and broker

Verifier and gate

THE QUESTION TO TAKE HOME

Who gave this process the authority?

github.com/zozo123/nyc-talk

Yossi Eliaz / Questions

Appendix / Reproduction

```
make demo      # Linux + bubblewrap; strict
make reference # policy checks; explicit skips
make deck      # evidence check + PDF build
```

29 checks passed. Source digests and subprocess observations are recorded.

Appendix / Limits of the evidence

Deterministic scripts exercise chosen actions.

Namespace and mount checks require the isolated run.

Five parser cases establish one demonstrated repair.

Controller code, keys and decisions remain trusted.

No cgroup, kernel-exploit, covert-channel or universal security claim.

Appendix / Primary references

Linux environ(7)

Environment inheritance

Docker bind mounts

Host filesystem effects and read-only mounts

RFC 9700

Access-token privilege restriction

SLSA artifact verification

Trusted provenance and artifact identity